

Prof. Stefan Göller
Anna Maiworm

Einführung in die Informatik

WiSe 2025/2026

Hausaufgabenblatt 11, Abgabe am 02.02.26 um 12:00 Uhr

Allgemeine Hinweise

- Wenn die Hausaufgabe exakte Benennungen von Dateien, Funktionen, Variablen, etc. vorgibt, dann müssen Sie sich an diese Vorgabe halten. Falls nicht wird Ihre Abgabe nicht oder nur mit Punktabzügen korrigiert.
- Gleiches gilt, wenn die Aufgabe konkrete Anweisungen für `input()` oder `print()` vorgibt.
- Ihr Programm darf keine zusätzlichen `print()` Befehle enthalten, welche in der Aufgabe nicht gefordert sind. Ist eine Ausgabe mit `print()` gefordert, so geben Sie, sofern nicht anders vorgegeben, nur das geforderte Ergebnis und **keinen** zusätzlichen Text aus.
- Benutzen Sie **keine Python-Imports**, außer dies ist in der Aufgabenstellung explizit erlaubt.
- Beachten Sie die Anweisungen, die ggf. zusätzlich in den jeweiligen Aufgaben gegeben sind.
- Geben Sie für jede Aufgabe eine separate Datei ab. Benennen Sie diese Datei als `bXXaY.py`, wobei XX die Blattnummer und Y die Aufgabennummer ist.
- Schreiben Sie die Namen aller Gruppenmitglieder als Kommentar in die erste Zeile jeder Datei.
- Programmcode muss möglichst einfach und gut lesbar sein, um die Korrektur zu erleichtern. Code, der nicht oder nur schwer lesbar ist, kann zu Punktabzug führen.

Allgemeiner Hinweis bezüglich Backtracking:

*Bei dem klassischen Ansatz für Backtracking nutzt man während des ganzen Programmablaufs **ein und dasselbe** Objekt das die Lösung darstellt. Diesem Objekt wird das nächste geratene Element hinzugefügt und nach dem rekursiven Aufruf für diesen Fall wird das geratene Element wieder aus dem Lösungs-Objekt entfernt (z.B. mittels `pop`, wenn das Lösungs-Objekt eine Liste ist und das geratene Element hinten angehängt wurde), bevor die nächste Alternative für das zu ratende Element betrachtet wird.*

*Es ist Ihnen aber auch gestattet stattdessen den folgenden Ansatz zu verfolgen: Immer wenn ein Element geraten wird, erstellt man einen **Klon** `k` des aktuellen Lösungs-Objektes `o`. Diesem wird das geratene Element hinzugefügt und im Anschluss wird `k` an den rekursiven Aufruf übergeben. In diesem Fall ist es also nicht notwendig, das geratene Element nach den rekursiven Aufruf wieder aus der Lösung entfernen, da dieses nur `k`, aber nicht `o` hinzugefügt wurde. Man kann also einfach mit der nächsten Alternative für das zu ratende Element weiter machen und für dieses genau so vorgehen.*

Sie dürfen selbst entscheiden welchen der zwei Ansätze Sie verwenden wollen. Falls Sie den klassischen Ansatz wählen sollten Sie, je nach Aufgabe, in Erwägung ziehen eine zusätzliche Hilfsfunktion zu schreiben, die das Entfernen des zuletzt geratenen Elements aus dem Lösungs-Objekt umsetzt.

Hausaufgabe 1 (1 + 2 + 5 = 8 Punkte):

Das Ziel dieser Aufgabe ist es, eine Funktion `zero_determinant` zu schreiben, welche eine Liste von Ganzzahlen `entries` erwartet und alle 3×3 -Matrizen, deren Einträge aus `entries` stammen und deren Determinante 0 ist berechnet.

In dieser Aufgabe sollen 3×3 -Matrizen als eine Liste der Länge drei dargestellt werden, wobei jeder der Einträge wiederum eine Liste der Länge drei ist, welche für eine Zeile der Matrix steht (angefangen mit der obersten Zeile).

- (a) Schreiben Sie eine Hilfsfunktion `det`, die eine Matrix `M`, wie oben beschrieben, erwartet und deren Determinante berechnet.

Hinweis: Die Determinante einer 3×3 -Matrix $M = \begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} \\ a_{1,0} & a_{1,1} & a_{1,2} \\ a_{2,0} & a_{2,1} & a_{2,2} \end{pmatrix}$ kann wie folgt berechnet werden:

$$\det(M) = a_{0,0} \cdot a_{1,1} \cdot a_{2,2} + a_{0,1} \cdot a_{1,2} \cdot a_{2,0} + a_{0,2} \cdot a_{1,0} \cdot a_{2,1} \\ - a_{2,0} \cdot a_{1,1} \cdot a_{0,2} - a_{2,1} \cdot a_{1,2} \cdot a_{0,0} - a_{2,2} \cdot a_{1,0} \cdot a_{0,1}$$

- (b) Schreiben Sie eine Hilfsfunktion **show**, die eine Matrix **M**, wie oben beschrieben, erwartet und diese als Rechteck von Zahlen, getrennt durch Leerzeichen, auf der Konsole ausgibt. Dabei sollen auch Matrizen, die Ganzzahlen unterschiedlicher Stelligkeit beinhalten, korrekt ausgerichtet sein (linksbündig je Spalte, siehe Beispiel). Außerdem soll im Anschluss eine Leerzeile auf der Konsole ausgegeben werden.

Beispiel: Führt man `show([[12,2,3],[4,514,6],[7,85,9]])` aus, so soll man die folgende Konsolenausgabe erhalten:

```
12 2   3
4  514 6
7  85  9
      <- Leerzeile
```

- (c) Schreiben Sie die Funktion **zero_determinant**, deren Funktionalität zu Beginn beschrieben wurde. Als Eingabe erwartet die Funktion also die Liste **entries**. **Alle** 3×3 -Matrizen mit den genannten Eigenschaften sollen mittels **Backtracking** berechnet und mit Hilfe Ihrer Funktion **show** auf der Konsole ausgegeben werden. Jede Lösung soll genau einmal ausgegeben werden, es darf also keine Lösung mehrfach ausgegeben werden.

Die Rückgabe der Funktion ist **None**.

*Hinweis: Wenn Sie `from copy import deepcopy` als erste Zeile in Ihr Programm einfügen können Sie `deepcopy(L)` verwenden, um einen Klon der Liste **L** zu erhalten, bei dem auch die inneren Listen geklont wurden (gleiche Funktionalität wie Blatt 7 Hausaufgabe 1 (b)).*

Hausaufgabe 2 (1 + 1 + 4 + 1 + 5 = 12 Punkte):

Das Ziel dieser Aufgabe ist es, eine Funktion **n_queens** zu schreiben, welche alle Lösungen des *n-Damen Problems* findet. Diese lautet wie folgt:

Gegeben eine positive Ganzzahl n , platziere n Damen auf einem $n \times n$ großen Schachbrett, sodass keine der Damen eine der anderen Damen schlagen kann.

Für die Dame gelten dabei die regulären Schachregeln, d.h. sie darf eine beliebige Distanz in eine beliebige Richtung (horizontal / vertikal / diagonal) ziehen, dabei jedoch keine Figuren überspringen.

Im Programm wird das Schachbrett als Liste der Länge n dargestellt, wobei jeder der Einträge wiederum eine Liste der Länge n ist, welche eine Zeile des Schachbretts darstellt.

Als Einträge für die Felder des Schachbretts (also Einträge der inneren Listen) nutzen wir 3 verschiedene Strings:

- "0" für ein leeres Feld
- "Q" für ein Feld mit Dame (Queen)
- "X" für ein Feld das von mindestens einer Dame bedroht wird

Wenn wir zum Beispiel $n = 5$ betrachten, so sehen das leere Schachbrett, und das Schachbrett mit einer an Position (2, 2) platzierten Damen, graphisch dargestellt, folgendermaßen aus:

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

X	0	X	0	X
0	X	X	X	0
X	X	Q	X	X
0	X	X	X	0
X	0	X	0	X

Entsprechen der Darstellung als Liste, befindet sich das Feld an Position (0, 0) oben links. Wenn wir im Folgenden von einer Position (i, j) sprechen, so bezieht sich i auf die Zeile und j auf die Spalte, in der sich das Feld befindet.

- Schreiben Sie eine Hilfsfunktion `show`, die ein Schachbrett `B`, wie oben beschrieben, erwartet und dieses als Rechteck von Zeichen, getrennt durch Leerzeichen, auf der Konsole ausgibt (siehe Beispiel in Aufgabenteil (e)). Außerdem soll im Anschluss eine Leerzeile auf der Konsole ausgegeben werden.
- Schreiben Sie eine Hilfsfunktion `initialize_board`, die die Dimension des Schachbretts, also die positive Ganzzahl n , erwartet. Zurückgegeben werden soll das „leere Schachbrett“ der Größe $n \times n$, also das Schachbrett in dem alle Einträge "0" sind.

Der Rumpf dieser Funktion darf nur aus **einer Zeile** bestehen.

Beispiel: Die Rückgabe von `initialize_board(3)` lautet
`[["0", "0", "0"], ["0", "0", "0"], ["0", "0", "0"]]`.

- (c) Schreiben Sie eine Hilfsfunktion `put_queen`, die ein Schachbrett `B` und eine Position `pos` (Tupel der Länge 2, wie oben beschrieben) erwartet. Die Funktion soll den Effekt haben, dass `B` eine Dame an Position `pos` hinzugefügt wird. Das heißt sowohl das "Q", als auch alle resultierenden "X" müssen eingetragen werden. Die Rückgabe der Funktion ist `None`.

Beispiel: Sei `B_bsp` das Schachbrett mit einer Dame an Position (2,2), aus dem Beispiel oben. Führt man `put_queen(B_bsp, (1,4))` aus, so stellt die im Anschluss unter `B_bsp` gespeicherte Liste das folgende Schachbrett dar:

X	0	X	X	X
X	X	X	X	Q
X	X	Q	X	X
0	X	X	X	X
X	X	X	0	X

- (d) Schreiben Sie eine Hilfsfunktion `free_fields`, die ein Schachbrett `B` und eine Ganzzahl `row` erwartet. Die Funktion überprüft für die Zeile `row`, in welchen der Spalten "0" eingetragen ist. Eine Liste dieser Ganzzahlen wird zurückgegeben.

Der Rumpf dieser Funktion darf nur aus **einer Zeile** bestehen.

Beispiel: Sei `B_bsp` das Schachbrett mit einer Dame an Position (2,2), aus dem Beispiel zu Beginn. Dann liefert `free_fields(B_bsp, 3)` die Rückgabe `[0,4]` und `free_fields(B_bsp, 2)` liefert die Rückgabe `[]`.

- (e) Schreiben Sie die Funktion `n_queens`, die das n -Damen Problem löst. Als Eingabe erwartet die Funktion die positive Ganzzahl n .

Die Funktion soll **Backtracking** verwenden um **alle** möglichen Lösungen zu finden und mit Hilfe Ihrer Funktion `show` auf der Konsole auszugeben. Starten Sie dabei mit dem leeren Schachbrett und versuchen Sie zunächst eine Dame in Zeile 0 zu platzieren, dann in Zeile 1, dann in Zeile 2 usw. Nutzen Sie dabei Ihre restlichen Hilfsfunktionen. Jede Lösung soll genau einmal ausgegeben werden, es darf also keine Lösung mehrfach ausgegeben werden.

Die Rückgabe der Funktion ist `None`.

Beispiel: Führt man `n_queens(5)` aus, so werden auf der Konsole die 10 Lösungen des 5-Damen Problems ausgegeben. Diese sehen wie folgt aus:

Q X X X X
X X Q X X
X X X X Q
X Q X X X
X X X Q X

Q X X X X
X X X Q X
X Q X X X
X X X X Q
X X Q X X

X Q X X X
X X X Q X
Q X X X X
X X Q X X
X X X X Q

X Q X X X
X X X X Q
X X Q X X
Q X X X X
X X X Q X

X X Q X X
Q X X X X
X X X Q X
X Q X X X
X X X X Q

X X Q X X
X X X X Q
X Q X X X
X X X Q X
Q X X X X

X X X Q X
Q X X X X
X X Q X X
X X X X Q
X Q X X X

X X X Q X
X Q X X X
X X X X Q
X X Q X X
Q X X X X

X X X X Q
X Q X X X
X X X Q X
Q X X X X
X X Q X X

X X X X Q
X X Q X X
Q X X X X
X X X Q X
X Q X X X

Hinweis: Beachten Sie auch hier den Hinweis bezüglich deepcopy aus Hausaufgabe 1.

Präsenzaufgaben

Präsenzaufgabe 1:

(a) Wir betrachten die folgende Problemstellung: Gegeben eine Liste positiver Ganzzahlen L und eine positive Ganzzahl `max_steps`, finde alle Wege durch die Liste L , die folgendes erfüllen:

- Der Weg beginnt an Position 0 und endet an der Position $n - 1$, wobei n die Länge von L ist. Position $n - 1$ darf nicht mehrfach besucht werden.
- Der Weg besteht aus höchstens `max_steps` vielen Zügen.
- Befindet man sich an Position p , so sind die erlaubten Züge x Positionen nach rechts oder links zu ziehen, wobei x ein Primteiler von $L[p]$ ist.
- Kein Zug geht jemals über die rechte oder linke Grenze der Liste hinaus.

Schreiben Sie eine Funktion `prime_divisor_walk`, welche **Backtracking** verwendet um **alle** solchen Wege auf der Konsole auszugeben.

Ein Weg soll dargestellt werden als Liste von Ganzzahlen, die die Züge angeben (positive Zahlen sind Züge nach rechts, negative Zahlen sind Züge nach links).

Beispiel: Führt man

```
prime_divisor_walk([4,4,4,4,4],6)
print("XXX")
prime_divisor_walk([4,4,4,4,4,4,4],2)
print("XXX")
prime_divisor_walk([2,2,2,2,1,2],5)
print("XXX")
prime_divisor_walk([14,74,46,86,29,45,91,34,90,24,10,52],5)
```

aus, so erhält man die folgende Konsolenausgabe (Reihenfolge der Lösungen irrelevant):

```
[2, 2]
[2, -2, 2, 2]
[2, -2, 2, -2, 2, 2]
XXX
XXX
XXX
[2, -2, 7, 2, 2]
[7, 2, 2]
[7, 2, -2, 2, 2]
[7, -2, 3, 3]
[7, -2, 5, -2, 3]
```

Schreiben Sie dazu zunächst die folgenden Hilfsfunktionen:

- Eine Funktion `is_prime`, welche eine Ganzzahl $i \geq 2$ erwartet und `True` zurückgibt falls i eine Primzahl ist, sonst `False`.
- Eine Funktion `prime_divisors`, welche eine positive Ganzzahl i erwartet und eine Liste aller Primteiler von i zurückgibt. Der Rumpf dieser Funktion darf nur aus **einer Zeile** bestehen.

- (b) Schreiben Sie eine Funktion `prime_divisor_walk_norep`, welche ebenfalls **Backtracking** nutzt und wie in Aufgabenteil (a) vorgeht, aber nur diejenigen Wege auf der Konsole ausgibt, die keine Position mehrfach besuchen.

Beispiel: Verwendet man das letzte Beispiel von zuvor und führt

```
prime_divisor_walk_norep([14,74,46,86,29,45,91,34,90,24,10,52],5)
```

aus, so erhält man als Konsolenausgabe nur noch die folgenden drei Wege (Reihenfolge der Lösungen irrelevant):

```
[7, 2, 2]
[7, -2, 3, 3]
[7, -2, 5, -2, 3]
```

Präsenzaufgabe 2:

Ziel dieser Aufgabe ist es, eine Funktion `word_search` zu schreiben, welche in einem übergebenen Buchstabengitter `G` ein bestimmtes Wort findet. Ein Buchstabengitter ist immer quadratisch, enthält ausschließlich Kleinbuchstaben und kann z.B. folgendermaßen aussehen:

g	a	r	n	g
w	a	h	a	n
g	u	r	n	r
a	n	r	n	a
r	v	b	m	g

Im Programm wird das Gitter als Liste von Listen von Strings der Länge 1 dargestellt, wobei die inneren Listen je für eine Zeile des Gitters stehen. Das Gitter von oben würde z.B. dargestellt werden als:

```
[["g", "a", "r", "n", "g"], ["w", "a", "h", "a", "n"],
 ..., ["r", "v", "b", "m", "g"]]
```

Das zu suchende Wort wird ebenfalls der Funktion übergeben und es sollen alle Vorkommen des Wortes im Gitter gefunden werden. Dabei sind alle geradlinigen Leserichtungen erlaubt, also vorwärts / rückwärts, horizontal / vertikal und auch diagonal (auch rückwärts).

- (a) Schreiben Sie eine Hilfsfunktion **show**, die ein Buchstabengitter **G**, wie oben beschrieben, erwartet und dieses als Rechteck von Buchstaben, getrennt durch Leerzeichen, auf der Konsole ausgibt. Außerdem soll im Anschluss eine Leerzeile auf der Konsole ausgegeben werden.
- (b) Schreiben Sie eine **rekursive** Hilfsfunktion **check_word**, die folgende Eingaben erwartet:
 - ein Buchstabengitter **G**, wie oben beschrieben
 - einen String **word** der Länge ≥ 1
 - ein Tupel **pos** der Länge 2, mit Einträgen zwischen 0 und $\text{len}(\mathbf{G})-1$, das für eine Position im Buchstabengitter steht: Der 1. Eintrag von **pos** gibt die Zeile an, der 2. Eintrag die Spalte. Entsprechend der Darstellung von **G** als Liste befindet sich Position (0,0) oben links.
 - ein Tupel **direction** der Länge 2, mit Einträgen aus $\{-1, 0, 1\}$, das die Leserichtung angibt: Der erste Eintrag von **direction** gibt an, ob das nächste zu lesende Zeichen in der nächsten Zeile (1), der gleichen Zeile (0) oder der vorherigen Zeile (-1) ist. Analog gibt der 2. Eintrag die Verschiebung der Spalte an. Zum Beispiel steht (1, -1) für die Richtung ↙.

Die Funktion **check_word** soll überprüfen, ob man im Gitter **G** von der Position **pos** aus, in Richtung **direction**, das Wort **word** lesen kann.

Ist dies der Fall, so soll das gefundene Wort durch Großbuchstaben hervorgehoben werden und das resultierende Gitter mittels Ihrer Funktion **show** auf der Konsole ausgegeben werden.

In jedem Fall soll **None** zurückgegeben werden.

Hinweis: Mittels `s.upper()` können Sie einen String `s` in Großbuchstaben überführen.

Beispiel: Sei **G_garn** das Buchstabengitter aus dem Beispiel oben. Führt man **check_word(G_garn, "garn", (0,4), (1,-1))** aus, so erhält man die folgende Konsolenausgabe:

```

g a r n G
w a h A n
g u R n r
a N r n a
r v b m g

<- Leerzeile

```

Führt man nach dem Definieren von `G_garn` zum Beispiel einen der drei folgenden Befehle aus, so erhält man keine Konsolenausgabe:

```

check_word(G_garn, "garn", (0,4), (1,1)),
check_word(G_garn, "garn", (0,0), (1,-1)),
check_word(G_garn, "GARN", (0,4), (1,-1))

```

- (c) Schreiben Sie die beschriebene Funktion `word_search`, die das zu durchsuchende Buchstabengitter `G` und das zu suchende Wort `word` erwartet. Die Funktion soll **Backtracking** verwenden um **alle** Vorkommen des Wortes im Gitter zu finden und die gefundene Lösung jeweils auf der Konsole auszugeben, d.h. das Gitter mit hervorgehobenem Wort, wie in Aufgabenteil (b) beschrieben. Die Rückgabe der Funktion ist `None`.

Beispiel: Führt man `word_search(G_garn, "garn")` aus, so werden werden die folgenden Lösungen auf der Konsole ausgegeben (Reihenfolge irrelevant):

G A R N g	g a r n G
w a h a n	w a h A n
g u r n r	g u R n r
a n r n a	a N r n a
r v b m g	r v b m g
G a r n g	g a r n g
w A h a n	w a h a N
g u R n r	g u r n R
a n r N a	a n r n A
r v b m g	r v b m G

Hinweis: Wenn Sie `from copy import deepcopy` als erste Zeile in Ihr Programm einfügen können Sie `deepcopy(L)` verwenden, um einen Klon der Liste `L` zu erhalten, bei dem auch die inneren Listen geklont wurden (gleiche Funktionalität wie Blatt 7 Hausaufgabe 1 (b)).

- (d) Schreiben Sie eine Funktion `word_search_mult_dir`, welche ebenfalls alle Vorkommen eines eingegebenen Wortes `word` in einem eingegebenen Buchstabengitter `G` finden und auf der Konsole ausgeben soll. Allerdings sind jetzt nicht nur die 8 zuvor beschriebenen geradlinigen Leserichtungen erlaubt, sondern nach jedem gelesenen Zeichen darf die Richtung gewechselt werden. Dabei darf jede Position aber nur ein mal besucht werden.

Hinweis: Die Funktion `check_word` kann hier zwar nicht unmittelbar als Hilfsfunktion verwendet werden, allerdings können Sie, um die Funktion `word_search_mult_dir` zu implementieren, einen Großteil des Codes aus den Aufgabenteilen (b) und (c) wiederverwenden und neu zusammensetzen.

Beispiel: Führt man `word_search_mult_dir(G_garn, "garn")` aus, so erhält man all die folgenden Lösungen:

G A R N g	G a R N g	g a r n G	g a r n g	g a R N g
w A h a n	w A h a n	w a h A n	w A h a n	w A h a n
g u r n r	g u r n r	g u R N r	G u r n r	G u r n r
a n r n a	a n r n a	a n r n a	A N r n a	a n r n a
r v b m g	r v b m g	r v b m g	R v b m g	r v b m g
G a r n g	g a r n G	g a r n G	g a r n g	g a r n g
w A h a n	w a h A N	w a h A n	w A h a n	w a h a N
g u R N r	g u r n R	g u R n r	G u R N r	g u r n R
a n r n a	a n r n a	a n r N a	a n r n a	a n r n A
r v b m g	r v b m g	r v b m g	r v b m g	r v b m G
G a r n g	g a r n G	g a r n G	g a r n g	g a r n g
w A h a n	w a h A n	w a h A n	w A h a n	w a h a n
g u R n r	g u r N R	g u R n r	G u R n r	g u r N R
a n r N a	a n r n a	a N r n a	a n r N a	a n r n A
r v b m g	r v b m g	r v b m g	r v b m g	r v b m G
G a r n g	g a r n G	g a R N G	g a r n g	g a r n g
w A h a n	w a h A n	w a h A n	w A h a n	w a h a n
g u R n r	g u r n R	g u r n r	G u R n r	g u r n R
a N r n a	a n r N a	a n r n a	a N r n a	a n r N A
r v b m g	r v b m g	r v b m g	r v b m g	r v b m G